

UNIVERSIDAD DEL VALLE DE GUATEMALA

DISEÑO DE LENGUAJES DE PROGRAMACIÓN

Proyecto 2: Generador YALex / YAPar

Construcción Autónoma de Analizadores Léxicos y Sintácticos

Integrantes del Grupo:

1. Angel Esteban Esquit Hernández - 23221
2. Javier Eduardo España Pacheco - 23361

Catedrático: Ing. Carlos Jorge Valdez Bautista

18 de mayo de 2026

Índice

1. Diseño del Software	2
1.1. Arquitectura General del Sistema	2
1.2. Flujo de Trabajo e Integración	2
2. Documentación y Explicación del Código	3
2.1. Generador de Analizadores Léxicos (<code>yalex_parser</code>)	3
2.2. Generador de Analizadores Sintácticos (<code>yapar_generator</code>)	3
3. Resolución de Casos de Prueba	4
4. Enlace al Repositorio de GitHub	4

1. Diseño del Software

El sistema **YALex Studio** es una suite integrada para la creación autónoma de compiladores, la cual consta de dos componentes modulares desarrollados completamente desde cero (sin dependencias externas de librerías de autómatas o motores de expresiones regulares):

- **Generador Léxico (YALex):** Construye analizadores léxicos (lexers) mediante el método directo a partir de archivos de especificación `.yal`.
- **Generador Sintáctico (YAPar):** Genera analizadores sintácticos (parsers) utilizando la técnica de análisis sintáctico SLR (Simple LR) sobre una colección canónica de items LR(0) a partir de archivos de especificación `.yapar`.
- **Independencia de los Analizadores Generados:** Los componentes resultantes (`lexer.py` y `parser.py`) son completamente autónomos e independientes de los módulos generadores. Esto permite su ejecución autónoma en cualquier entorno de Python estándar sin requerir la presencia de la base de código del compilador.

1.1. Arquitectura General del Sistema

El proyecto está estructurado de manera modular y desacoplada, separando la lógica de análisis, simulación, generación de código y la interfaz de usuario:

```
PRY2-DLP/
+- src/
| +- yalex_parser/          <- Implementacion original del lexer
| | +- parser.py           <- Parser de especificaciones .yal
| | +- regex_parser.py     <- Parser de regex y AST
| | +- direct.py          <- AFD por metodo directo
| | +- dfa.py              <- Minimizacion de AFD
| | +- simulator.py        <- Simulacion con traza
| | +- codegen.py          <- Generacion del Lexer
| +- yapar_generator/      <- Generador de parsers LR(0)/SLR
| | +- yapar_parser.py     <- Parser de especificaciones .yapar
| | +- lr0.py              <- Automata LR(0), closure y transiciones
| | +- table.py            <- Generador de tablas SLR
| | +- parser.py           <- Maquina de pila de analisis
| | +- codegen.py          <- Generacion del Parser
| +- common/               <- Utilidades de tokens
| +- bridge_cli.py         <- Interfaz Bridge
| +- bridge_yapar.py       <- Integration YAPar
+- desktop-app/            <- Interfaz de usuario (Tauri + React)
+- examples/               <- Casos de prueba (Low, Medium, High)
```

1.2. Flujo de Trabajo e Integración

El flujo para la integración del Lexer y el Parser sigue los siguientes pasos:

1. **Entrada YALex (.yal):** Se procesa el archivo de tokens, construyendo el AFD correspondiente para generar un script Python ejecutable (`lexer.py`).
2. **Entrada YAPar (.yapar):** Se procesa la especificación sintáctica, calculando los conjuntos de items LR(0), las transiciones y la tabla SLR para generar un parser autónomo (`parser.py`).
3. **Ejecución en Cadena:** El archivo de código de entrada (`input.txt`) es tokenizado secuencialmente por el lexer generado, y el flujo de tokens alimenta directamente al parser SLR, que produce el árbol de análisis sintáctico (AST) o reporta los errores sintácticos con precisión.

2. Documentación y Explicación del Código

2.1. Generador de Analizadores Léxicos (`yalex_parser`)

Este módulo es el responsable de analizar la especificación de tokens y construir la lógica del escáner:

- `parser.py`: Lee la definición del archivo `.yal`, separando la sección de definiciones (`let`), las reglas (`rule`) y el código empotrado del usuario (`header` y `trailer`).
- `regex_parser.py` y `regex_ast.py`: Traducen la notación de las expresiones regulares a un árbol de sintaxis abstracta (AST) autónomo. Soporta operaciones de cerradura de Kleene (`*`), unión (`|`), concatenación, y rangos (como `[a-z]`).
- `direct.py`: Utiliza el método directo para calcular las funciones de árboles de expresiones regulares: `nullable`, `firstpos`, `lastpos` y `followpos`. Convierte el AST de la regex directamente en un Autómata Finito Determinista (AFD).
- `dfa.py`: Aplica el algoritmo de Hopcroft para minimizar los estados del AFD generado, garantizando eficiencia óptima durante la tokenización.
- `simulator.py`: Contiene el motor que evalúa los caracteres de entrada frente al AFD minimizado utilizando una lógica de *Maximal Munch* (máxima coincidencia).

2.2. Generador de Analizadores Sintácticos (`yapar_generator`)

Este módulo implementa la lógica de análisis LALR/SLR basada en la gramática libre de contexto (CFG) provista:

- `yapar_parser.py`: Analiza el archivo de gramática `.yapar`. Reconoce tokens declarados con `%token`, el símbolo inicial con `%start`, y la lista de producciones sintácticas junto con sus acciones semánticas asociadas.
- `lr0.py`: Modela los items LR(0) ($A \rightarrow \alpha \cdot \beta$), estados del autómata, y calcula recursivamente el `closure` de conjuntos de items y las transiciones mediante la función `GOTO`. Esto genera el Autómata LR(0) Canónico.

- **table.py**: Computa las funciones auxiliares de gramáticas **FIRST** y **FOLLOW** para todos los no-terminales. A partir de los **FOLLOW** sets y el autómata LR(0), construye la tabla de análisis sintáctico SLR (filas de estados, columnas de terminales y no-terminales) con acciones de:
 - **SHIFT (s)**: Desplazar un terminal al tope de la pila y transicionar a un nuevo estado.
 - **REDUCE (r)**: Reducir una regla de producción aplicando la correspondiente substitución del no-terminal.
 - **ACCEPT (acc)**: Aceptación exitosa de la cadena de entrada.
 - **ERROR**: Estado en blanco en la tabla que representa un error sintáctico.
- **parser.py**: Implementa el motor de ejecución basado en pila (Stack Machine) que simula el autómata SLR, manteniendo el registro exacto del historial de pasos (Trace) para propósitos de depuración.

3. Resolución de Casos de Prueba

El sistema fue validado rigurosamente utilizando tres escenarios de prueba de complejidad incremental ubicados en la carpeta **examples/**:

- **Calculadora Simple (low)**: Valida expresiones aritméticas de sumas y restas utilizando tokens de identificadores y números.
- **Lenguaje Estructurado (medium)**: Valida flujos de control simples como condicionales **if-else**, bucles **while**, asignaciones y llamadas a funciones.
- **Lenguaje OOP de Alta Complejidad (high)**: Representa un subconjunto de lenguaje similar a Java con clases, herencia, constructores, excepciones y arreglos.

4. Enlace al Repositorio de GitHub

El código fuente completo de la implementación, incluyendo el generador de autómatas, el creador de tablas SLR, el parser guiado por pila, las pruebas unitarias y los ejemplos de entrada se encuentra alojado en el siguiente repositorio público de GitHub:

<https://github.com/Javier-Espana/PRY2-DLP>